ENTERPRISE

Inventory Management System

Organisation : Infosys Springboard

Full-Stack Web Application | Academic Project

Presented by: Mahmmadsami M Bilaggi, Batch 13

Information Science Department

Table of Contents

Project Overview & Navigation

01 Introduction & Project Overview

02 Problem Statement

03 Why Build This Project?

04 Technology Stack & Roles

05 System Architecture & Diagram

06 Layered Architecture Breakdown

07 Controller Layer

08 Service Layer & Transaction Module

09 Simulation Module & Execution Flow

10 Project Achievements & Outcomes

11 Real-World Applications

12 Conclusion & Thank You

Introduction

Enterprise Inventory Management System

Project Overview

- A robust, full-stack Enterprise Inventory Management System built with Java Spring Boot and Thymeleaf that streamlines business operations.
- Handles end-to-end inventory lifecycle: product creation, stock tracking, product management, stock tracking, reporting and dashboard analytics.
- Designed for real-world enterprise use with MVC architecture, layered service design, and database integrity.
- The system simulates realistic business scenarios including stock depletion, reorder triggers, and risk threshold alerts.

MVC
Architecture Pattern

Spring MVC
API Design Style

ACID
Transaction Compliance

OOP
Java Design Paradigm

Problem Statement

Challenges in Traditional Inventory Management

Manual Tracking Errors

Spreadsheet-based tracking leads to inaccurate stock counts, data inconsistency, and human errors causing costly overstock or stockouts.

No Real-Time Visibility

Businesses lack real-time stock level monitoring, resulting in delayed decisions, undetected shortages, and missed reorder triggers.

Fragmented Systems

Separate tools for procurement, sales, suppliers, and reporting create data silos with no centralized control or audit trail.

No Risk Threshold Alerts

Without automated threshold logic, managers are unaware of critical inventory levels until operations are already disrupted.

Poor Transaction Integrity

Non-ACID transactions result in partial updates, orphaned records, and data corruption across multi-step inventory operations.

Scalability Limitations

Legacy systems fail to scale with growing product catalogs, multiple warehouses, and increasing transaction volumes.

Why Build This Project?

Motivation, Goals & Academic Value

1

Industry-Relevant Tech Stack

Spring Boot, Thymeleaf, JPA, and MySQL are widely adopted in enterprise Java development. This project provides hands-on expertise in production-level frameworks.

2

Solve a Universal Business Need

Every business — retail, manufacturing, healthcare — requires inventory control. Building this system addresses a universal operational challenge.

3

Practice Layered Architecture

Implementing Controller → Service → Repository layers enforces clean separation of concerns — a fundamental software engineering principle.

4

Learn Transactional Integrity

Applying @Transactional annotations and handling rollback scenarios builds understanding of ACID-compliant database operations in practice.

5

Build End-to-End Features

From authentication to simulation modules and risk alerts, this project integrates full-cycle development: backend, frontend, database, and business logic.

6

Academic & Career Portfolio Value

A fully functional enterprise application demonstrates senior-level development skills and provides concrete evidence of system design capabilities for employers.

Technology Stack

Tools, Frameworks & Libraries Used

Language

Java 17

Core programming language

Backend Framework

Spring Boot

Auto-configured REST/MVC

Frontend Engine

Thymeleaf

Server-side HTML templating

Web Layer

Spring MVC

HTTP routing & controllers

ORM Provider

Hibernate

Entity-relational mapping

Database

MySQL

Relational data persistence

ORM Layer

Spring Data JPA

Database abstraction layer

UI Layer

HTML/CSS

Bootstrap-styled frontend

Role of Each Technology

How Every Component Contributes to the System

Java 17 Foundation Language	Provides OOP constructs, streams, lambdas, and generics. All business logic, entities, and services are implemented in Java.
Spring Boot Application Backbone	Auto-configures the server, manages beans via IoC, provides embedded Tomcat, and enables rapid development without XML config.
Spring MVC Request Routing	@Controller classes handle HTTP requests, map URL paths, validate inputs, and return Thymeleaf views.
Spring Data JPA Data Access Layer	Repository interfaces extend JpaRepository, providing CRUD and custom query methods without boilerplate code.
Thymeleaf View Rendering	Server-side template engine that binds model attributes from controllers to HTML pages, enabling dynamic content rendering.
MySQL Persistent Storage	Stores all entities: products, orders, stock movements, and user data in relational tables with foreign-key integrity.
Hibernate ORM Engine	Translates Java entity classes to database tables, manages entity states, and generates SQL queries automatically.
@Transactional Data Integrity	Wraps multi-step operations in atomic database transactions. Ensures rollback on failure, preventing partial state corruption.

Architecture Type

MVC + Layered N-Tier Architecture

Model–View–Controller (MVC) with 4-Layer N-Tier Architecture

User Browser

PRESENTATION LAYER

Thymeleaf Views (HTML Templates)

CONTROLLER LAYER

Spring MVC @Controller

SERVICE LAYER

@Service + @Business Logic

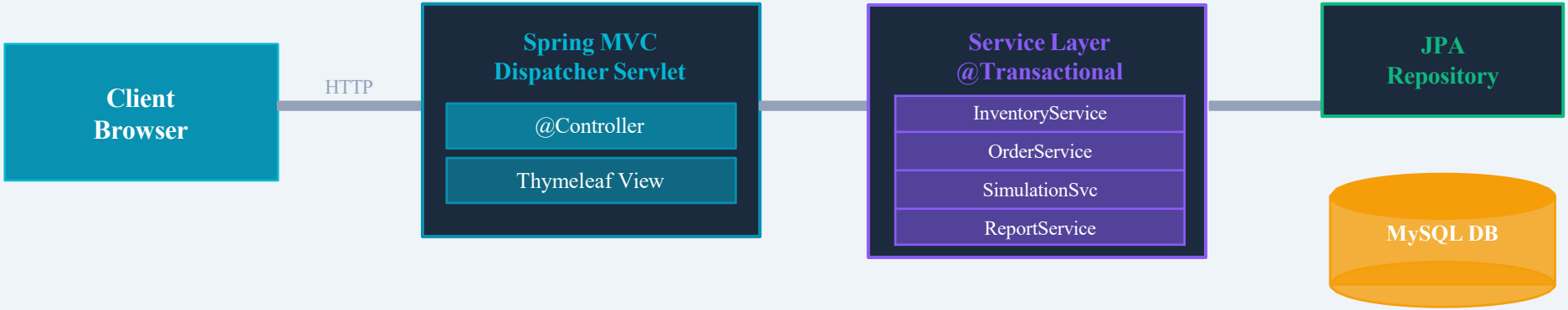
DATA ACCESS LAYER

Spring Data JPA Repositories + Hibernate + MySQL

MySQL Database

System Architecture Diagram

End-to-End System Communication Flow



Core Application Modules

Layered Architecture

Separation of Concerns Across All System Layers

Presentation Layer
Thymeleaf Templates

- product.html, add-product.html
- Bootstrap 5 responsive UI
- Thymeleaf fragments for reuse
- Model attributes bound from controllers

Controller Layer
@Controller / @RestController

- Handles HTTP GET/POST requests
- Calls service methods
- Adds model attributes to view
- Input validation via @Valid

Service Layer
@Service + @Transactional

- Contains all business logic
- Orchestrates multi-step operations
- Manages transaction boundaries
- Simulation & risk threshold logic here

Repository Layer
JpaRepository + Hibernate

- Extends JpaRepository<Entity, ID>
- Custom @Query with JPQL/SQL
- Hibernate manages entity lifecycle
- Direct MySQL interaction via JDBC

Controller Layer

Spring MVC Controllers — Request Handling & Routing

Role of Controllers

@Controller	Returns view name for Thymeleaf templates, adds model data
@RequestMapping	Maps URL paths to handler methods (GET/POST)
@PathVariable	Extracts dynamic path segments like /product/{id}
@ModelAttribute	Binds form data to Java objects automatically
@Valid	Triggers Bean Validation on form-submitted objects
BindingResult	Captures validation errors without throwing exceptions
RedirectAttributes	Passes flash messages across redirects

AuthController

GET /login — show login page
GET /register — user registration
GET /logout — logout user
Purpose : Handles authentication and user access

InventoryController

GET /dashboard — main dashboard
GET/products — view all products
POST /add-product — add new product
POST /increase/{id} — increase stock by 1
POST /decrease/{id} — decrease stock by 1

StockController /stock

GET /stock — view all stock
POST /stock/adjust — manual adjust
GET /stock/alerts — low stock list

Service Layer

Business Logic, Transaction Management & Orchestration

@Service

Marks class as a Spring-managed service bean. Detected by component scan and injected via @Autowired.

@Transactional

Wraps entire method in a database transaction. Rolls back on RuntimeException ensuring data consistency.

Business Logic

Centralized domain rules: price calculation, discount logic, stock deduction, reorder point evaluation.

Service Interfaces

Services implement interfaces for loose coupling, testability, and easy mocking in unit tests.

Key Service Methods & Logic

processOrder(OrderDTO dto)	Validates stock → deducts quantity → creates order record → updates stock ledger → triggers alert if below threshold. Fully transactional.
restockProduct(int id, int qty)	Adds stock to product → logs stock movement → clears any low-stock alert → recalculates total inventory value.
evaluateRiskThreshold(Product p)	Compares currentStock against minimumThreshold. Returns CRITICAL/WARNING/OK enum. Called after every stock-deducting operation.
generateInventoryReport()	Aggregates product values, calculates total inventory worth, identifies top movers and critical stock items for dashboard display.

Transaction Service Module

ACID Compliance, @Transactional Behavior & Rollback Strategy

A

Atomicity

All steps in a transaction succeed or none commit. If order save fails, stock deduction is rolled back.

C

Consistency

Database moves from one valid state to another. Stock never goes negative; order is never created without stock deduction.

I

Isolation

Concurrent transactions don't interfere. Spring uses default READ_COMMITTED isolation preventing dirty reads.

D

Durability

Committed transactions survive crashes. MySQL InnoDB ensures durability via write-ahead logging.

Order Processing Transaction Flow

1. BEGIN Transaction
2. Validate Stock
3. Deduct Stock
4. Create Order
5. Update Ledger
6. COMMIT or ROLLBACK

Execution Flow

Step-by-Step Request → Response Journey Through the System

1

User Action

User submits 'Create Order' form in browser. HTTP POST /orders/create with form data.

2

DispatcherServlet

Spring's front controller intercepts the request, identifies matching @RequestMapping in OrderController.

3

Controller Processes

OrderController validates @ModelAttribute OrderDTO via @Valid. On error → returns form view with BindingResult.

4

Service Invoked

Controller calls orderService.processOrder(dto). Spring opens @Transactional context here.

5

Business Logic

Service checks stock availability, deducts quantity, creates Order entity, updates StockMovement, calls threshold evaluator.

6

Repository Saves

JPA repositories persist Order + StockMovement entities. Hibernate generates INSERT/UPDATE SQL on MySQL.

7

Transaction Committed

All DB writes committed atomically. If any step threw RuntimeException, full rollback would have occurred.

8

Response Rendered

Controller adds success flash message via RedirectAttributes, redirects to /orders — Thymeleaf renders updated order list.

Why This Approach?

Design Decisions & Justification

Why Spring Boot?	Spring Boot eliminates XML configuration boilerplate, auto-configures embedded Tomcat, and provides production-ready features (actuator, security) out of the box — significantly reducing development time.
Why Thymeleaf over React/Angular?	For an enterprise MVC application, Thymeleaf integrates naturally with Spring model attributes. No separate frontend build pipeline needed. Server-side rendering ensures SEO and immediate page consistency.
Why Layered Architecture?	Strict separation of Controller/Service/Repository layers enforces Single Responsibility Principle. Each layer can be unit tested independently, and business logic changes don't ripple to the presentation layer.
Why @Transactional for All Operations?	Inventory operations inherently involve multiple DB writes (stock + order + ledger). Without atomicity, a server crash mid-operation would leave data in an inconsistent state — unacceptable in enterprise systems.
Why a Simulation Module?	Real-world systems are tested under load before production. The simulation module validates system behavior under concurrent order processing and ensures threshold logic fires correctly at scale.

Project Achievements & Outcomes

What Was Delivered & What Was Learned

7+
Core Modules Delivered

100%
ACID-Compliant Transactions

3
User Roles (RBAC)

4
Architecture Layers

Technical Skills Gained

- Spring Boot auto-configuration mastery
- JPA entity relationships (OneToMany, ManyToOne)
- @Transactional rollback handling
- Thymeleaf fragment templating

System Features Delivered

- Full CRUD for all inventory entities
- Real-time stock alert dashboard
- Simulation engine with logging
- Role-based access control

Engineering Principles Applied

- SOLID principles in service design
- DRY via utility classes & fragments
- Separation of Concerns in layers
- Fail-fast validation at controller

Real-World Applications

Industry Use Cases for This System

Retail & E-Commerce

Track SKUs across multiple outlets, manage reorder points, sync online and in-store stock in real time to prevent overselling.

Manufacturing

Monitor raw material inventory, track work-in-progress stock, manage finished goods, and coordinate with suppliers for JIT procurement.

Healthcare & Pharmacy

Track medical supply expiry dates, manage critical stock thresholds for life-saving medicines, automate procurement of low-stock items.

Warehousing & Logistics

Multi-location stock tracking, shipment management, inbound/outbound logging, and automated space optimization alerts.

Restaurant & F&B

Manage perishable ingredient inventory, track daily consumption patterns, set critical thresholds for essential ingredients.

IT Asset Management

Track hardware/software assets, manage procurement cycles for devices, and maintain audit trails for compliance reporting.

Conclusion

Summary, Learnings & Future Roadmap

Project Summary

The Enterprise Inventory Management System successfully demonstrates the development of a production-grade, full-stack Java application using Spring Boot and Thymeleaf. The system implements industry-standard architectural patterns (MVC, Layered Architecture), enforces ACID-compliant transactional integrity, and delivers a comprehensive suite of modules covering the full inventory lifecycle — from product management and supplier coordination to simulation testing and risk threshold monitoring.

This project has successfully met all academic objectives while laying a strong foundation for real-world enterprise inventory system deployment.

Thank You!

Enterprise Inventory Management System

Questions & Discussion Welcome